



Pontificia Universidad Católica del Ecuador

Ingeniería de Software

Tarea



Autor: Joshua Castañeda

Docente: Ing. JOSE NARANJO.

Asignatura:

PRÁCTICAS EN GESTIÓN DE LA INFORMACIÓN

Manabí – Ecuador

Julio 2026



Conexión a Base de Datos desde C#/.NET: SqlConnection, SqlCommand y SqlDataReader

1. Definición y Propósito

El acceso a bases de datos en aplicaciones desarrolladas con C# y .NET se realiza comúnmente mediante ADO.NET, un conjunto de clases del framework .NET diseñadas para establecer conexiones, ejecutar consultas y manipular información almacenada en sistemas gestores de bases de datos (SGBD) como SQL Server.

Problema que resuelve: las aplicaciones necesitan una forma estandarizada, segura y eficiente de comunicarse con una base de datos relacional sin depender directamente del motor específico ni de código nativo. ADO.NET actúa como una capa intermedia que traduce las operaciones del programa (leer, insertar, actualizar, eliminar registros) en comandos que el servidor de base de datos puede ejecutar, devolviendo los resultados de forma estructurada.

Importancia: prácticamente toda aplicación empresarial necesita persistir y consultar datos. Dominar ADO.NET es la base para entender tecnologías de más alto nivel (Entity Framework, Dapper, etc.), ya que estas últimas terminan apoyándose internamente en las mismas clases: SqlConnection, SqlCommand y SqlDataReader.



Entre las clases más importantes de ADO.NET se encuentran:

- SqlConnection
- SqlCommand
- SqlDataReader

2. Componentes Principales

2.1 SqlConnection

Es la clase encargada de establecer y administrar la conexión entre la aplicación y una base de datos SQL Server. Su función principal es proporcionar un canal de comunicación seguro mediante una cadena de conexión que contiene información como el servidor, la base de datos y las credenciales necesarias para el acceso.

- Sin una conexión activa, la aplicación no puede ejecutar consultas ni interactuar con los datos almacenados.
- Administra los recursos asociados a la comunicación con el servidor (sockets, sesiones, etc.).
- Se recomienda utilizar la instrucción using, ya que garantiza la liberación automática de recursos una vez finalizada la operación, incluso si ocurre una excepción.



2.2 SqlCommand

Representa una instrucción SQL que será enviada al servidor para su ejecución. Permite ejecutar consultas de selección (SELECT), inserción (INSERT), actualización (UPDATE) y eliminación (DELETE), así como procedimientos almacenados.

Dependiendo del tipo de operación, SqlCommand ofrece distintos métodos de ejecución:

Método	Uso principal	Retorna
ExecuteReader()	Consultas que devuelven filas de resultados (SELECT)	Un SqlDataReader
ExecuteNonQuery()	Operaciones que modifican datos (INSERT, UPDATE, DELETE)	Número de filas afectadas
ExecuteScalar()	Consultas donde se espera un único valor (ej. COUNT, MAX)	Un solo valor (object)



2.3 SqlDataReader

Se utiliza para leer los resultados de una consulta SQL de forma secuencial y eficiente (forward-only, read-only). Es especialmente adecuada cuando se necesita recorrer grandes cantidades de registros, ya que:

- Mantiene una conexión abierta con la base de datos mientras se leen los datos.
- Recupera los datos fila por fila, reduciendo el consumo de memoria frente a cargar todo el resultado en memoria de una sola vez.
- Se obtiene mediante el método ExecuteReader() de un objeto SqlCommand.
- El método Read() permite avanzar registro por registro, mientras que el acceso a los valores de cada columna se realiza por índice (reader[0]) o por nombre de campo (reader["Nombre"]).

Relación entre los componentes

Los tres componentes trabajan de forma encadenada y dependiente entre sí, ninguno funciona de manera aislada: SqlCommand necesita una SqlConnection abierta para ejecutarse, y SqlDataReader depende de que el SqlCommand que lo generó siga vinculado a una conexión activa mientras se están leyendo los datos.



3. Flujo de Trabajo

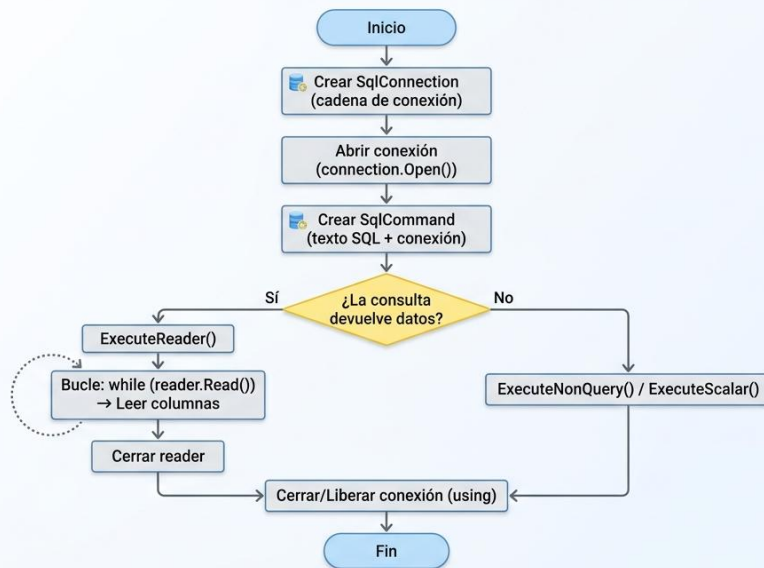
Pasos típicos para implementar el acceso a datos con estas tres clases:

1. Definir la cadena de conexión (servidor, base de datos, credenciales o autenticación integrada).
2. Crear y abrir la conexión con `SqlConnection`, idealmente dentro de un bloque `using`.
3. Crear el comando (`SqlCommand`) con la instrucción SQL y asociarlo a la conexión abierta.
4. Agregar parámetros al comando (recomendado, para evitar inyección SQL) si la consulta los requiere.
5. Ejecutar el comando con el método adecuado (`ExecuteReader`, `ExecuteNonQuery` o `ExecuteScalar`).
6. Leer los resultados con `SqlDataReader.Read()` en un bucle, si corresponde.
7. Cerrar y liberar recursos (automático si se usó `using`).



Diagrama de flujo secuencial

DIAGRAMA DE FLUJO: PROCESO DE CONEXIÓN Y CONSULTA A SQL SERVER (C#)



4. Ventajas y Desventajas

Ventajas

- Rendimiento: al ser una API de bajo nivel, tiene muy poco overhead comparado con ORMs.
- Control total sobre la consulta SQL exacta que se ejecuta.
- Eficiencia de memoria con SqlDataReader, ya que no carga todo el resultado en memoria.



- Es la base sobre la que se construyen tecnologías más avanzadas (Entity Framework, Dapper), por lo que entenderla facilita aprender esas herramientas.

Desventajas

- Código más extenso y repetitivo (boilerplate) comparado con un ORM.
- Mayor responsabilidad del desarrollador: manejo manual de apertura/cierre de conexión, mapeo de columnas a objetos, y prevención de inyección SQL.
- SqlDataReader es de solo lectura y solo avance (forward-only): no se puede retroceder ni modificar datos directamente sobre él.
- Requiere mantener la conexión abierta mientras se lee, lo que puede ser menos flexible en escenarios donde se necesita desconectar rápido el recurso.

5. Casos de Uso

- Aplicaciones de escritorio o servicios que requieren máximo rendimiento en consultas frecuentes o sobre grandes volúmenes de datos (ej. sistemas de facturación, reportes en tiempo real).
- Procesos batch o ETL que leen millones de registros secuencialmente, donde el bajo consumo de memoria de SqlDataReader es crítico.
- APIs backend donde se necesita ejecutar procedimientos almacenados existentes en la base de datos corporativa.



- Capas de acceso a datos (DAL) personalizadas en proyectos donde no se desea (o no se permite) usar un ORM completo.
- Migraciones o scripts de mantenimiento que ejecutan sentencias SQL puntuales de forma controlada.

6. Mejores Prácticas

Recomendaciones

- Usar siempre bloques using (o try/finally) para garantizar el cierre de SqlConnection, SqlCommand y SqlDataReader.
- Utilizar parámetros (SqlParameter / Parameters.AddWithValue) en lugar de concatenar texto SQL, para prevenir inyección SQL.
- Mantener las conexiones abiertas el menor tiempo posible (abrir justo antes de ejecutar, cerrar justo después de leer).
- Aprovechar el connection pooling que ADO.NET gestiona automáticamente: no es necesario implementarlo manualmente, solo evitar mantener conexiones abiertas innecesariamente.
- Cerrar explícitamente el SqlDataReader antes de reutilizar la misma conexión para otra consulta.



Errores comunes a evitar

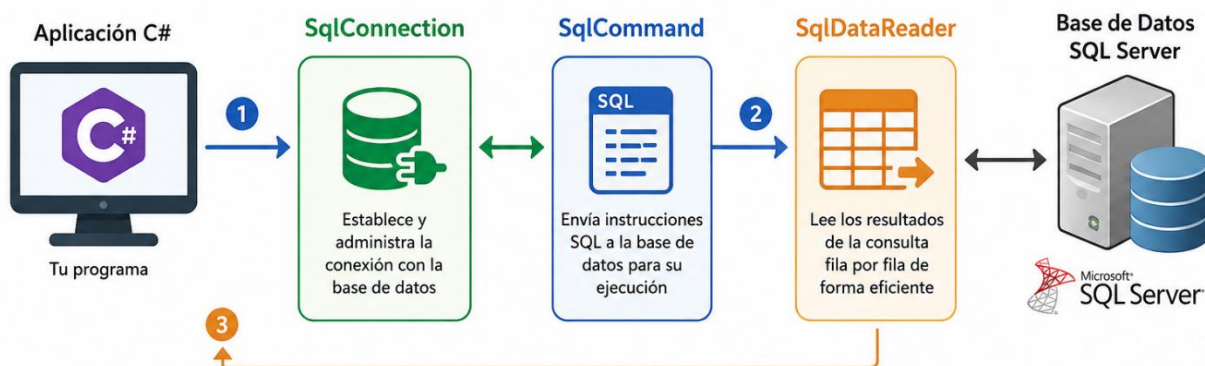
- Concatenar valores de usuario directamente en el texto SQL (riesgo de inyección SQL).
- Olvidar cerrar la conexión o el reader, lo que agota el pool de conexiones disponibles.
- Intentar acceder a los datos del SqlDataReader después de cerrarlo o de cerrar la conexión asociada.
- Usar ExecuteReader() cuando en realidad solo se necesita un valor (ExecuteScalar) o el número de filas afectadas (ExecuteNonQuery), lo cual añade overhead innecesario.

Consejos de rendimiento

- Usar SqlDataReader (en vez de cargar todo en un DataSet/DataTable) cuando se procesan grandes volúmenes de datos de forma secuencial.
- Especificar explícitamente el tipo y tamaño de los parámetros SQL cuando sea relevante, para mejorar el uso de planes de ejecución en el servidor.
- Evitar ejecutar consultas dentro de bucles (problema N+1); preferir una sola consulta bien diseñada o el uso de procedimientos almacenados.
- Considerar comandos asíncronos (ExecuteReaderAsync, ExecuteNonQueryAsync) en aplicaciones con alta concurrencia para no bloquear hilos mientras se espera respuesta del servidor.



Relación entre SqlConnection, SqlCommand y SqlDataReader



- 1 SqlConnection:** La aplicación crea un objeto SqlConnection y lo abre para conectarse a la base de datos.
- 2 SqlCommand:** Con la conexión abierta, se crea un objeto SqlCommand que contiene la instrucción SQL (SELECT, INSERT, UPDATE, DELETE, etc.) y se envía al servidor para su ejecución.
- 3 SqlDataReader:** Si la instrucción devuelve resultados (por ejemplo, un SELECT), ExecuteReader() devuelve un objeto SqlDataReader que permite leer los datos fila por fila. Cuando termina la lectura, se cierra el DataReader y la conexión queda disponible para nuevas operaciones.

